

GE M02 Classes, Objects, Methods, Decisions, Test Cases and Algorithms

Points: 25 (See Rubric in Canvas)

Due Date: Due date listed in Canvas but some sections will be due as class participation before. Create Calendar reminders

- Final submission will be accepted up to 24 hours after the due date with a 10% penalty. Meaning if you turn it in at 12:01 am of the next day you will be deducted 10% of the total points from your score.
- If the assignment is more than 24 hours late, it will be a 0.

Submission: Upload files separately and do not upload a zip file. **Only use concepts that have been covered in class lectures so far.**

- **Two Document Files as PDF or Word document:**
 - The GE M02 document with your answers in the highlighted areas
 - [Technical Document](#)
- **Three Java Files (. java files NOT CLASS FILES) for**
 - [2.1 Strings and Chars Madlib](#)
 - [2.2 M02 L05 BMI Lab Program](#)
 - [2.3 Concert Ticket Prices](#)

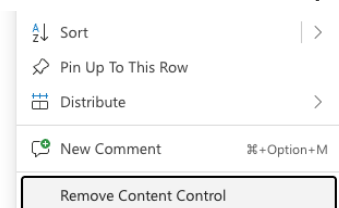
Objectives:

- Analyze and apply module 01 concepts
- Develop effective technical documentation to demonstrate your understanding of technical concepts, with a focus on clarity and professional presentation.
- Apply version control and debugging practices using Eclipse and GitHub.

Effort: You are encouraged to collaborate to discuss concepts and explore writing code together. Remember

- Review lectures and links in the lectures to information first.
- Follow [CS Academic Integrity and AI Policy - Harding](#)
- If you find you don't understand the information in the lecture reach out to **Deb, Elaine, Heriberto, Sayuj or Alizah.**

If you download this as a word document and you have any problems typing your answer in the tables you must click on 3 dots and remove content control.



[GE M02 Classes, Objects, Methods, Decisions, Test Cases and Algorithms](#)

[Part 1: Technical Document Explore and Explain](#)

[Part 2 Analyze and Apply](#)

[2.1 Strings and Chars Madlib](#)

[2.2 M02 L05 BMI Lab Program](#)

[2.3 Concert Ticket Prices](#)

[3 Learnings and Reflection](#)

Part 1: Technical Document Explore and Explain

Purpose: Professional communication to demonstrate your learnings by connecting different ideas from lectures or provided resources.

- Clearly explain key ideas in your own words and include supporting evidence such as code examples, images, screenshots, or charts.

Update your technical document for Module 2.

- Use headings to organize topics and create or update a table of contents to be able to quickly access information.
 - [Headings, Subheadings, and Table of Contents \(Google Docs\)](#)
 - [Headings, Subheadings, and Table of Contents \(Microsoft Word\)](#)
- Add more comments to your GuessNumber.java code from GE M01 explaining concepts from GE M02 and add this example to your technical document.
- Complete [Module 2: Predefined Classes, Methods, and Decision Structures](#) section by adding information for the objectives and organizing in sections using headings..

Part 2 Analyze and Apply

Work with others to discuss and solve problems to write code. You are not graded on correctness but on practicing concepts from lectures.

- Do not use someone else's or AI's answers or code.
- Come to office hours if you are having problems writing the code.

2.1 Strings and Chars Madlib

Write a Java program that creates a short Mad Lib story.

This program focuses on reading input correctly and building output using Strings and characters.

- `next()` → reads one word (stops at whitespace)
- `next().charAt(0)` → reads one character

We are not using `nextLine()` for this assignment.

Program Requirements:

1. Use your code you started in lecture M02L06MathCharsStrings
2. Prompt the user and read the following as Strings and store in variables:
 - A name
 - A noun
 - A verb ending in -ing
 - An adjective
 - A place

```
Enter a first name: Sayuj
Enter a last name: Shrestha
Enter a noun: burrito
Enter a verb ending in -ing: sitting
Enter an adjective: funny
Enter a place: Olympics
```

3. Read a Character

```
Enter your favorite letter: Q
```

4. Extract the first character of the name and store it in a char.
5. Build the Mad Lib Story (Concatenation) by createing a single String that combines:
 - a. the user's words
 - b. the extracted characters
 - c. Sentence words
 - d. punctuation and spaces
6. Print the string out

```
Once upon a time, Sayuj S was sitting through the Olympics with a burrito. It
was a very funny day. Their favorite letter was Q."
```

7. Add comments to
 - a. Explain the difference between declaring a String and a char data type.

A string is essentially a daisy chain of multiple chars in a row, Really a distinguishable difference Beyond the fact that one is essentially a subset of another but have different methods due to the variety in their use case

- b. Explain how to read in a string and how to read in a char.
Essentially in memory both strings and jars are stored with numerical values and the computer reads these numerical values as they are stored hence why commands like "charAt" Work for strings. Essentially, all it's doing is telling the computer to look within the string of chars and pick out the specific value at the designated coordinates and store it in a variable
 - c. Explain what charAt(0) is doing.
Essentially what Char at zero is doing is looking for the first try in the daisy chain of a string and returning that value to whatever variable it's being called to
 - d. Explain what concatenate means and how you do it.
8. Upload your java source file when you submit and make sure you put it in your github remote repository. Update your technical documentation with your code examples containing comments.

2.2 M02 L05 BMI Lab Program

Open M02 L05 to review the BMI calculation lab you started in class. You should have set up your Eclipse Project for CS1050M02 and added the BodyMassIndex.java file.

Create a program to calculate your Body Mass Index and display the correct category. [BMI Calculator Body Mass Index](#)

- Use US units: $bmi = weight (lb) \div height^2 (inches) * 703$
- Display correct category using

CDC Adult BMI Categories	
BMI	Status
Under 18.5	Underweight
18.5 to under<25	Normal
25 to under < 40	Overweight
40 or greater	Obese

Example output

```
This program will calculate your body mass index (BMI)
Enter your weight in pounds: 150.5
Enter your height in inches: 66
```

Weight: 150.5 Height: 66.00
Your body mass index (BMI) is 24.3
Your category: normal

1. Create test data to make sure you test each weight category. Review M02L07 test cases for the BMI program.

Pre-Condition	Algorithm Step	Post Condition
100.00 lbs 67in Bmi 15.66	The steps for each test case are the same run the variables for weight and height through the calculator using BMI_US_FACTOR As the only set constant	Underweight
150.5 lbs 66 inches Bmi 24.3	Program runs the calculations and then checks against if statements to see if it is within the appropriate category for the number in which case it would outprint that the user has a normal BMI and then I'll print what their bmi value is	Normal
167 lbs 67in Bmi 26.15	Programmable rent a check and make sure the end total is within acceptable range for the calculation in this case would tell the user that they are in the overweight category and out print their BMI	Overweight
350 lbs 67in Bmi 54.81		Obese
35lbs 67in Bmi 5.48	In the program exists as the last else condition in the if else if the user bmi is under the normal range	Underweight

2. Write your algorithm on paper.

```
// Establish scanner class for later interface with user
```

```
//Established three doubles for the separate categories of what classifies as overweight, standard and obese
```

```
Cons double
```

```
Cons double
```

Cons double

Cons double

```
//Create a print statement in store user inputted variable to use in later math
```

```
User double
```

```
User double
```

```
//Do the calculations with user inputs behind the scene but don't print them yet
```

```
//The filter numerical operations through categorical Cliff statements using the established constants for over-under and obese categories to designate different responses based on what the results are as well as print the BMI for user review
```

Insert your algorithm written on paper here

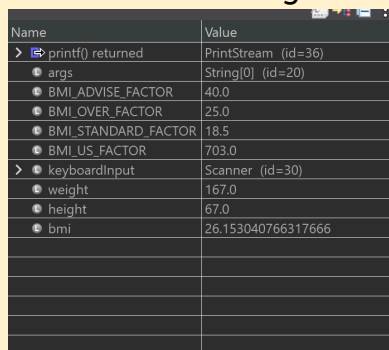
3. What if/else algorithm did you choose to display the correct category status and why?

I staggered the if else categories starting with the first if Case being the obese category given that it was the highest numerical value and then tapering off down to underweight from there

Why can't you use a switch statement?

There were too many moving variables for switch case statement to be effective here if the values were more hard-set than in this case would be practical but the switch case wouldn't be able to cover as many results in as little code

4. Implement a little bit of code and test. Run the debugger to show your test data will step into the correct condition. Put a screenshot showing the variables while using the debugger.



The screenshot shows a debugger window with two columns: 'Name' and 'Value'. The 'Name' column lists variables and expressions, and the 'Value' column shows their current values. The variables include 'args', 'BMI_ADVISE_FACTOR', 'BMI_OVER_FACTOR', 'BMI_STANDARD_FACTOR', 'BMI_US_FACTOR', 'keyboardInput', 'weight', 'height', and 'bmi'. The values are: 'args' is 'PrintStream (id=36)', 'BMI_ADVISE_FACTOR' is '40.0', 'BMI_OVER_FACTOR' is '25.0', 'BMI_STANDARD_FACTOR' is '18.5', 'BMI_US_FACTOR' is '703.0', 'keyboardInput' is 'Scanner (id=30)', 'weight' is '167.0', 'height' is '67.0', and 'bmi' is '26.153040766317666'.

Name	Value
> printf() returned	PrintStream (id=36)
• args	String[0] (id=20)
• BMI_ADVISE_FACTOR	40.0
• BMI_OVER_FACTOR	25.0
• BMI_STANDARD_FACTOR	18.5
• BMI_US_FACTOR	703.0
> • keyboardInput	Scanner (id=30)
• weight	167.0
• height	67.0
• bmi	26.153040766317666

5. Add comments to your code so you can use this in your technical document. Commit and push. Put a screenshot showing your code in your github remote repository.

2.3 Concert Ticket Prices

Determine the ticket price for one seat at a concert where you

- Create test cases for precondition and postconditions
- Write an algorithm on paper
- Implement a little bit of code and test after you complete sections below

Requirements of what the program must do.

1. Prompt the user for a character that represents the seating section. Use these characters and section names (store as char, not String):
 - **F**: Front Section
 - **C**: Center Section
 - **S**: Side Section
 - **U**: Upper Section
2. Use `Character.toUpperCase(someChar)`; to convert the inputted value to upper case before determining if it is invalid.
3. If an invalid section is entered, print a message and end the program.
Example: When user inputs X, display “Invalid section, program ending”
4. **After the section character is read and verified**, display the full section name.
Example: input C → output Selected Center Section
5. **Prompt the user for the row number** (integer between 1 and 60).
 - If an invalid row is entered, print a message and end the program.
Example: When user inputs 0 or 75, display “Invalid row, program ending”
6. Seat pricing depends only on row:

Row 1–15 cost \$450
Row 16–30 cost \$300
Row 31–60 cost \$200
7. **Output the result** showing section, row, and price.

Example Outputs

Example #1: Valid Input

Enter ONE character for section - F: Front C: Center S: Side U: Upper

C

Selected Center Section

Enter row

22

Section C row 22 price: \$300

Example #2: Invalid Section

Enter ONE character for section - F: Front C: Center S: Side U: Upper

Z

Invalid section, program ending

Example #3: Invalid Row

Enter ONE character for section - F: Front C: Center S: Side U: Upper

S

Selected Side Section

Enter row

65

Invalid row, program ending

1. In **two or three sentences**, restate the problem in your own words (inputs, outputs, rules).

Received two possible user inputs and compare them to constant values so that the program can report the appropriate price for user selection(s)

2. Identify **data types** needed for each input and for price:

Section: char

Row: int

Price: int

3. Evaluate Section Values that are inputted by the user

3.1 Create at least **2 more** test cases covering at least

- 1 valid inputs one upper case and one lower case example
- 1 invalid input

Precondition	Algorithm Step	Postcondition
Valid Input: f	Make upper case If F, C S or U valid input so next prompt for row	Selected Front Section

Invalid Input: 2	else not F, C S or U valid end program	Invalid section, program ending
Valid input: 1-60	Store value and then run through if statements for price based on row compared to set set prices	print appropriate response with row and section listed along with price
Invalid input 1>x<60	Check input against if statements and if the result does not fit print error invalid row	End Program

3.2 What would be best to use: **nested if** or **multi-way if** or **switch** and why
A multi-way if statement would be the best way to go for this given scenario given the variables and the subsection categories for prices and sections it is important to have the variety that a multi-way if would allow

3.3 Write an algorithm on paper for evaluating the section based on your learnings from the test cases. Take a picture and insert below.

If F || C || S || U = userInput

Print "Please enter row"

3.4 Create a new class called ConcertTicketPrices. Implement a few lines of code and test.

4. Evaluate Row Values that are inputted by the user to determine price. When this is entered they will already have a correct Section entered. Assume the user always enters a number.

4.1 Create at least **4** more test cases covering valid/invalid rows:

- Valid Value Row 16-30 cost \$300
- Valid Value Row 31-60 cost \$200
- Invalid Value Row less than 1
- Invalid Value Row less than greater than 60

Precondition	Postcondition	Code Notes
Section f Valid row:1	Section F row 1 price: \$450	If rows 1-15 cost \$450
Section f Valid row:17	Section F row 1 price: \$300	Sort user statement into algorithmic categories for price and if it falls

		within range apply the appropriate cost which currently would be \$300
Section f Valid row:32	Section F row 1 price: \$200	Cross check user input with if statement conditionals and apply the appropriate price which would be \$200
Section f inValid row:-1	ERROR invalid row end program	If condition is lower than set parameters throw an error message and tell the user the program is ending
Section f inValid row:61	ERROR invalid row end program	The condition is higher than the set parameters throw an error message and tell the user the program is ending

4.2 Explain the algorithm **nested if** or **multi-way if** or **switch** you think will work best for determining the price.

A multi-way if works best in the circumstance because we have three separate conditions that we need to check and give a response to if they meet them along with One additional category for if none of the three conditions are met

4.3 Write an algorithm on paper for evaluating the row based on your learnings from the test cases. Take a picture and insert below.

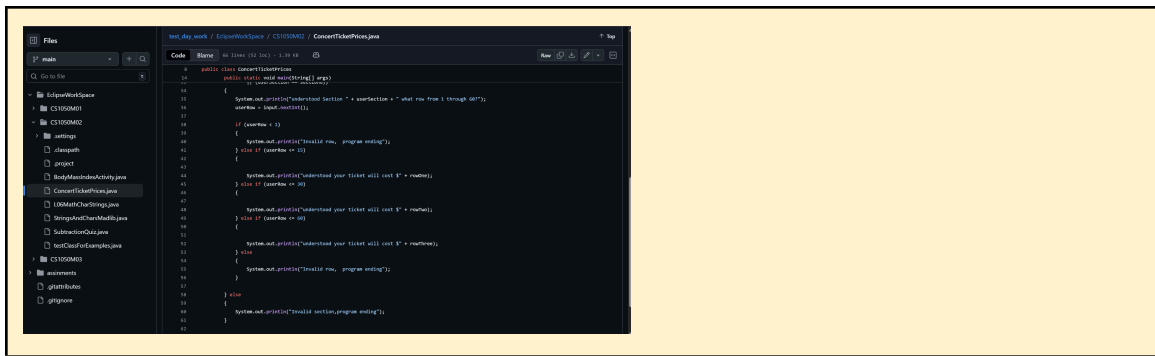
If $x \leq 1$
price

Else If $x \leq 15$
price2

Else If $x \leq 31$
price2

4.4 Implement a few lines of code and test.

5. Take a screenshot of your code on the github server repository and insert below.



3 Learnings and Reflection

1. Explain the purpose of writing test cases - precondition and postcondition. To work out mentally how the code might operate before we metaphorically put pen to paper

2. Explain two things from module 1 that you were able to understand better when writing your code for this assignment. Switch cases were particularly interesting to me because I've seen them before but have never actually used them and didn't quite know how to properly implement them

3. Explain a moment you got "stuck" and how you worked through it. Reiterating my code especially for the ticket prices program was more difficult than I thought I was able to get functioning code but refining it in ways that made sense was difficult because I kept trying to implement new methods but running into walls logically as I was writing down test cases

4. Explain two things you didn't understand from a lecture in module 02 that you understand better after completing the guided exploration. Switch cases and The difference between multi-way if and Switch cases were confusing to me mostly due to a general lack of understanding about switch cases which made me confused about what the use difference was

5. Explain two things you have been doing outside of class that are helping you understand the concepts better.

Reading up where I can and revisiting old coding projects I used to work on to re-establish the basics of my understanding and different logical processes I might need to use in the future when programming

